

Tuning Our Algorithm

As mentioned, simulated annealing is a sensitive process. The code we've introduced shows how to do it in a basic way, but we'll want to make changes to the details in order to do better. This process of changing small details or parameters of an algorithm in order to get better performance without changing its main approach is often called *tuning*, and it can make big differences in difficult cases like this one.

Our `perturb()` function makes a small change in the itinerary: it switches the place of two cities. But this isn't the only possible way to perturb an itinerary. It's hard to know in advance which perturbing methods will perform best, but we can always try a few.

Another natural way to perturb an itinerary is to reverse some portion of it: take a subset of cities, and visit them in the opposite order. In Python, we can implement this reversal in one line. If we choose two cities in the itinerary, with indices `small` and `big`, the following snippet shows how to reverse the order of all the cities between them:

```
small = 10
big = 20
itinerary =
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29,
 \30, 31, 32, 33, 34, 35, 36, 37, 38, 39]
itinerary[small:big] = itinerary[small:big][::-1]
print(itinerary)
```

When you run this snippet, you can see that the output shows an itinerary with cities 10 through 19 in reverse order:

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 19, 18, 17, 16, 15, 14, 13, 12, 11, 10, 20, 21,
 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39]
```

Another way to perturb an itinerary is to lift a section from where it is and place it in another part of the itinerary. For example, we might take the following itinerary:

```
itinerary = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

and move the whole section `[1, 2, 3, 4]` to later in the itinerary by converting it to the following new itinerary: